requires computing and comparing $D(v, A_o)$ and $D(v, A_i), i \in 1, 2$ for *all vectors* in the dataset. To minimize the cost, LIRE only examines nearby postings for reassignment check by selecting several A_o's nearest postings, over which two condition checks were applied to generate the final reassign set. Experiments in Section 5 show empirically that only a small number of nearby postings for the two necessary condition checks is enough to maintain the index quality.

After obtaining vector candidates for reassignment, LIRE executes the reassignment. For vector candidate v, LIRE first searches v's new closest posting , then performs NPA check to get rid of false-positives: if a vector actually does not need reassignment, the reassign operation is aborted. Otherwise, LIRE appends v in the newly identified posting that is NPA-compliant and then deletes v in the original posting.

3.4 Split-Reassign Convergence

In this section we prove that a split-reassign action to the vector index, despite the potential of triggering cascading split-reassign actions, will converge to a final state and terminate in finite steps. We first formally define the states of vector index and the events triggering state transitions. Then we prove that state transition will converge and terminate.

Index State: The state of a vector index for a vector data-set V comprises of two parts.

$$C : \text{set of posting centroids.} \tag{3}$$

$$M : \text{vector membership to centroid(s).} \tag{4}$$

According to LIRE, given C, each vector in M is assigned to its nearest centroid in C, i.e., M is *uniquely* determined by C.

Index-State Transitions: The state transition is triggered by two types of events:

$$E_{insert} : \text{a vector } v \text{ is inserted into the vector index.} \tag{5}$$

$$E_{delete} : \text{a vector } v \text{ is deleted into the vector index.} \tag{6}$$

A reassign of vector v is considered as an E_{delete} of v followed by an E_{insert} of v. Note that an event will change the state of M, however it would not necessarily alter the state of C.

Also note that only an event E_{insert} may incur a *split action* of the vector index, which alters the state of C and subsequently M. Since C uniquely determines M, we can only focus on the state change of C.

Split-Reassign Convergence Proof: E_{delete} may eventually trigger a merge during a search process (according to LIRE), and the merge obviously will terminate.

Suppose an E_{insert} triggers a sequence of changes of C, denoted as C_i,C_{i+1}, ..., C_{i+N}. To prove the convergence is to show that N is a finite number.

We note that C has the following properties:

- $|C| \leq |V|$: The cardinality of C is bounded and no greater than the cardinality of V, i.e., the vector dataset.

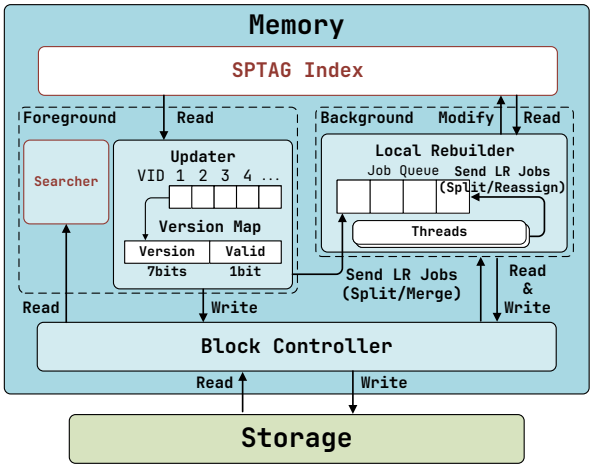

Figure 5. SPFresh architecture (LR means Local Rebuild).

- $|C_{i+1}| = |C_i|+1$: Each split action will delete an old centroid from C_i, and adds two new centroids to it. Therefore, the cardinality of C *always* increases by one per split action.

Based on Property 2, $|C_{i+N}| = |C_i| + N$. And according to Property 1, $N \leq V - |C_i|$ because $|C_{i+N}| \leq V$. Since $|V|$ is finite, N must also be finite. Therefore the split action must terminate in finite steps. □

4 SPFresh Design and Implementation

4.1 Overall Architecture

Figure 5 shows the system architecture of SPFresh. SPFresh reuses the SPANN SPTAG index (depicted in Figure 3) for fast posting centroid navigation as well as its searcher to serve queries. It further introduces three new modules to implement LIRE, namely, a light-weight *In-place Updater*, a low-cost *Local Rebuilder*, and a fast storage *Block Controller*.

Updater appends a new vector at the tail of its nearest posting and maintains a *version map* to keep track of vector deletion by setting a corresponding tombstone version to prevent deleted vectors from appearing in the search results. The map is also used to trace the replica of each vector. By increasing the version number, it marks the old replicas as deleted. The system keeps a global in-memory version map and stores vectors along with the version number on disk. A vector is stale if the in-memory version number is greater than that on the disk. This can be used for garbage collection caused by reassignment. The use of version can defer and batch the garbage collection so as to control the I/O overhead of vector removal. After vector insertions are completed in-place, the Updater checks the length of the posting and then sends a split job to *Local Rebuilder* if the length exceeds the split limit. The actual data deletions are performed asynchronously as a batch during local rebuild phase when the posting length exceeds the limit.

Local Rebuilder is the key component to implement LIRE. It maintains a job queue for split, merge, and reassign jobs and dispatches jobs to multiple background threads for concurrent execution.

- A *split job* is triggered by *Updater* when a posting exceeds the split limit. It cleans deleted vectors in the oversized posting and splits it into small ones if needed.
- A *merge job* is triggered by the *Searcher* if it finds some postings are smaller than a minimum length threshold. It merges nearby undersized postings into a single one.
- A *reassign job* is triggered by a split or merge job, which rebalances the assignment of vectors in the nearby postings.

When the background split and merge jobs are complete, SPFresh will update the memory SPTAG index with the new posting centroids to replace the old one.

Block Controller serves posting read, write, and append requests, as well as posting insertion and deletion operators on disk. It uses the raw block interface of SSD directly to avoid unnecessary read/write amplification incurred by some general storage engines, such as Log-structured-merge-tree-based KV store. Each posting may span multiple SSD blocks, each of which stores multiple vectors (including vector ID, version ID, and raw data). The *Block Controller* also maintains an in-memory mapping from the posting ID to its used SSD blocks as well as the free SSD blocks pool.

Next, we will discuss the design and implementation of *Local Rebuilder* (§4.2) and *Block Controller* (§4.3) in detail.

4.2 Local Rebuilder Design

In order to move split, merge, and re-assign jobs off the update critical path, SPFresh divides the update process into two parts, a foreground *Updater* and a background *Local Rebuilder*. These two components form a feed-forward pipeline, where *Updater* is the producer of requests to the *Local Rebuilder*. In this pipeline, the background *Local Rebuilder* is a key module that implements merge, split, and reassign operators of LIRE protocol efficiently to keep up with the foreground *Updater*.

4.2.1 Rebuild Operators of LIRE protocol.

Local Rebuilder implements LIRE with three basic operators.

Merge: To execute a merge job, the *Local Rebuilder* simply follows the merge protocol described in §3.2.

Split: After receiving an oversized posting split job, the *Local Rebuilder* first garbage collects deleted vectors in the posting and verifies whether the posting length after garbage collection still exceeds the split limit. If not, the *Local Rebuilder* writes the garbage-collected posting back to storage and completes the split job.

Otherwise, a *balanced clustering process* is triggered to split the oversized posting into two smaller ones. In particular, *Local Rebuilder* leverages the multi-constraint balanced clustering algorithm in [67] to generate high-quality centroids and balanced postings.

After splitting, *Local Rebuilder* puts two new postings back to the index and deletes the original oversized postings.

Reassign: A reassign job is generated by merge or split jobs. It checks if vectors in the new postings and/or their neighbors need to be relocated to re-balance the data distributions in the local region. The reassignment check is based on the two *necessary conditions* in §3.3. Note that neighbor posting check is *not* required for merge-triggered reassign.

Reassigning a vector without deleting its replicas in the unexamined postings increases the replica number. This not only increases storage overheads but also increases split and reassign frequency since the extraneous replicas take up spaces of postings. In order to efficiently identify stale vectors after reassignment without actual deletes, *Local Rebuilder* uses a version map to record *the version number* for each vector. A version number takes one byte and is stored in memory to record the version changes of a vector: seven bits for re-assign version and one bit for deletion label. When reassigning a vector, we increase its version number in the version map and append the raw vector data with its new version number to the target posting. All the old replicas with a stale version number are dropped during the search. The replicas will be garbage collected later.

4.2.2 Concurrent Rebuild.

In SPFresh, *Local Rebuilder* is multi-threaded with efficient concurrency control of updates to the in-memory and on-disk data structures. Concurrent rebuild can avoid drops of index quality due to slow re-balance.

Concurrency Control for Append/Split/Merge: Since append, split, and merge may update the same posting and the in-memory block mapping concurrently, We add a fine-grained posting-level write lock between these three operations to ensure a posting change is atomic.

Posting read does not require a lock. Therefore, identifying vectors for reassignment is lock-free since it only searches the index and checks the two necessary conditions. Our experiments show that even in a skewed workload, write lock contention is low, i.e., less than 1% contention cases. This is because only a small portion of postings are being edited concurrently.

During a reassign process, it is possible that a vector appends to a stale posting, which happened to be deleted concurrently. In such a case, we abort the reassignment and re-execute the reassign job for this vector. In our experiments, there are only less than 0.001% of total insertion requests encountering the posting-missing problem caused by split. As a result, the abort and re-execution overhead is minor.

Concurrent Reassign: SPFresh avoids concurrently reassigning the same vector at the same time. When collecting vector candidates for reassignment, *Local Rebuilder* gathers the current version of the candidates. *Local Rebuilder* atomically executes reassignment operations by leveraging atomic primitives of compare-and-swapping (CAS) for the version

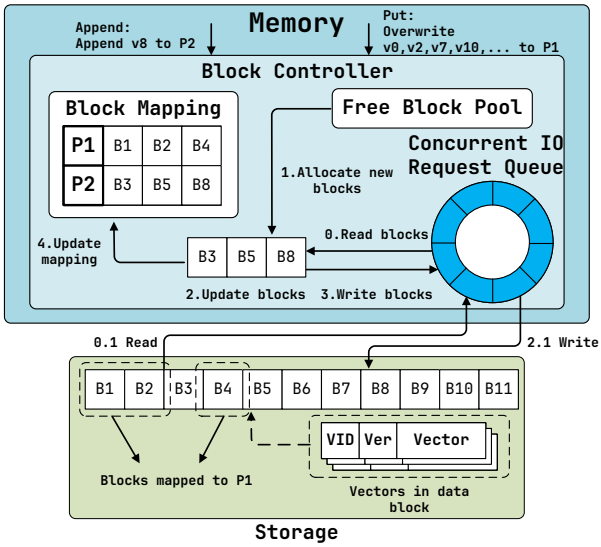

Figure 6. SPFresh storage overview.

number. If an atomic CAS operation fails on the vector version map, reassignment is aborted since the vector becomes a stale version. Otherwise, we let the corresponding reassign proceed to the end.

4.3 *Block Controller* Design

Block Controller is a light-weight storage engine highly optimized for reading. It offers *append-only* operation on postings. This design takes advantage of the characteristics of postings, where old posting data is immutable, and the update introduces no additional overheads for reading (unlike a log-structured file system, where multiple additional out-of-place reads are required). It keeps appending vector updates to a posting before exceeding a length limit. When a posting exceeds the length limit, the old posting is destroyed after being split into two new ones. To avoid unnecessary overheads in file systems or other storage engines (e.g., KV store), *Block Controller* operates directly on raw SSD interfaces.

SPDK-based Implementation: Figure **??** overviews the storage design. *Block Controller* is implemented on top of SSD. It leverages the raw *block* interface offered by SPDK [25], a high-performance NVMe SSD library by Intel. SPDK offers a set of user-space IO libraries for accessing high-speed NVMe devices, which allow us to bypass the legacy storage stack to perform SSD I/Os directly.

Storage Data Layout: As shown in Figure **??**, a *Block Controller* consists of in-memory *Block Mapping*, a *Free Block Pool*, and a *Concurrent I/O Request Queue*.

Block Mapping maps a posting ID to its SSD block offsets. Since the posting ID is a continuous integer, block mapping is implemented as an in-memory dense array, where each element stores the block metadata of a posting length and its SSD block offsets. A posting consists of a list of tuples in the form of <vector id, version number, raw vector>, which typically takes three to four SSD blocks. A block mapping entry only consumes 40 bytes of memory. For one billion vectors, there only exist 0.1 billion postings. In this case, block mapping only consumes about 4GB of memory.

Free Block Pool maintains all free SSD blocks. It keeps track of the offsets of all the free blocks to serve disk allocation, and garbage collects stale blocks after spilt and reassign.

Concurrent I/O Request Queue is implemented using an SPDK circular buffer, which sends asynchronous read and write to SSD device for maximized IO throughput and low I/O latency.

Posting API & Implementation: *Block Controller* provides a set of posting APIs as follows:

- `GET` retrieves posting data by the given ID. The request first looks up the block mapping to identify the corresponding SSD blocks. Asynchronous I/Os are then sent to the current I/O Request Queue. Later, all desired blocks are collected upon the completion of all I/Os.
- `ParallelGET` reads multiple postings in parallel to amortize the latency of individual `GET`s. This ensures fast search and update. `ParallelGET` allows sending a batch of I/O requests to fetch all the candidate postings, which hides the I/O latency and boosts disk utilization.
- `APPEND` adds a new vector to a posting's tail. Instead of read-modify-write at the posting-granularity, `APPEND` only involves read-modify-write of the last *block* of a posting, which reduces the amount of read/write amplification significantly. As shown in Figure **??**, `APPEND` first allocates a new block, reads the original last block if the last block is not full, appends new values to the values from the last block, and then writes it as a new block. After a new block is written, it atomically updates the corresponding in-memory *Block Mapping* entry via a compare-and-swap operation to reflect the change. The old block will be released to the free block pool for later usage.
- `PUT` writes a new posting to SSD. Like `APPEND`, it allocates new blocks and writes for the entire posting blocks in bulk. Then it atomically updates the *Block Mapping* entry. If `PUT` overwrites an old posting, it releases old blocks to the *Free Block Pool*.

Block Controller provides a common abstraction and implementation, which can be generalized for other read-intensive applications (such as widely-used inverted index for search engine [2, 11]).

4.4 Crash Recovery

SPFresh adopts a simple crash recovery solution, which combines snapshot and write-ahead log (WAL). Specifically, an index snapshot is taken periodically, and all update requests between adjacent snapshots are collected into a WAL so that a crash can be recovered from the latest snapshot, followed

by replaying the WAL. The WAL will be deleted when a new snapshot is generated.

To take a snapshot for a vector index, we need to record both the in-memory and on-disk data structures. For in-memory index data, we create snapshots for centroid index, version maps in *Updater*, and block mapping and block pool in *Block Controller*, and flush the snapshots to disk. Snapshots are relatively cheap because these data structures take only 40GB for billion-level dataset, which costs 2~3 seconds for a full flush on a PCI-e based NVM SSD. For disk data, thanks to our block-level copy-on-write mechanism, we can collect all the released blocks during two snapshots into a *pre-release* buffer, which will be added to the *Free Block Pool* after the next snapshot is recorded. Thus all the data blocks modified in the interim can be rolled back to be consistent with the previous snapshot. This solution saves a large amount of disk space since we only delay the space release for old blocks during two snapshots.

5 Evaluation

In this section, we conduct experiments to answer the following questions:

- How does SPFresh compare with state-of-the-art baselines in terms of performances, search accuracy and resource usage? (§5.2)
- What is the maximum performance of SPFresh? (§5.3)
- Can SPFresh solve the data shifting problem illustrated in Figure 2? (§5.4)
- How to properly configure SPFresh? (§5.5)

5.1 Experimental Setup

Platform: All experiments run on an Azure lsv3 [41] VM instance, which is a storage-optimized virtual machine with locally attached high-performance NVMe storage. In particular, we configured the VM with 16 vCPUs from a hyper-threaded Intel Xeon Platinum 8370C (Ice Lake) processor and 128GB memory for our experiments.

Datasets: We use two widely-used vector datasets to evaluate SPFresh:

- SIFT1B [40] is a classical image vector dataset for evaluating the performance of ANNS algorithms that support large-scale vector search. It contains one billion of 128-dimensional byte vectors as the base set and 10,000 query vectors as the test set.
- SPACEV1B [1] is a dataset derived from production data from commercial search engines. It represents a different form of vector encoding: deep natural language encoding. It contains one billion of 100-dimensional byte vectors as a base set and 29,316 query vectors as the test set.

Baselines: We compare SPFresh with two baselines:

- *DiskANN* is the state-of-the-art disk-based fresh ANNS system [53]. It is based on a graph ANNS index and uses an out-of-place update solution. We configure DiskANN with the same settings as in their paper [53]. For update configurations, DiskANN baseline processes *streamingMerge*, a lightweight global graph rebuild, for every new 30M vectors, where graph degree R equals 64 and insert candidate list equals 75. For search configurations, DiskANN baseline uses the default setting with beamwidth equal to 2 and search candidate list L equal to 40 for recall10@10.
- *SPANN+*, a modified version of SPANN [67] which appends updates locally to a posting *without splitting and reassigning*. This is an *append-only* version of SPFresh without the *Local Rebuilder* module.

Workloads: Three workloads are used in the experiments.

- *Workload A* simulates a realistic vector update scenario with 100 million scale of SPACEV vectors. The reason to reduce the scale from 1 billion to 100 million is that DiskANN requires several TBs of DRAM to run 1-billion scale fresh updates (shown in Table 1), which exceeds our machine's capacity. In particular, workload A simulates 1% update daily over 100 days. To generate updates realistically, we extract two disjoint SPACEV 100M datasets from SPACEV1B, where one is used as the base ANNS index data-set and the other as the update candidate pool. Each daily update epoch deletes 1% of vectors randomly from the base ANNS index, and inserts 1% of vectors randomly selected from the update data pool to the base index.
- *Workload B* has the same data scale and sampling method as Workload A but with a 100 million scale of the SIFT vector dataset.
- *Workload C* scales up our experiment data-set to be *billion-scale* using both the SIFT dataset and the SPACEV dataset. This workload aims at stress testing SPFresh, also with a 1% daily update rate.

Metrics: SPFresh is designed for online ANNS streaming scenarios. Thus, our evaluation focuses on the following four categories of metrics.

- Search Performance: We measure tail (P90, P95, P99, and P99.9) latency and query per second (QPS) throughput. In particular, we have a hard cut of 10ms for SPFresh and all baselines, where the system finishes the result immediately and returns the current search results.
- Search Accuracy: We use the percentage of ground truths recalled by SPFresh system to measure accuracy.
- Update Performance: insertion and deletion throughputs.
- Resource Usages: the memory and CPU consumption.

5.2 Real-World Update Simulation

In this experiment, we compare all the metrics of SPFresh with all the baselines on the real-world situation. We use Workload A and B (see §5.1) to simulate 100 days of real-world updates, and we show that SPFresh outperforms baselines in all evaluation metrics.